

Quality Stack

Revision: 1 Last modified: 2026-06-06T00:00:00Z

`docker-compose.quality.yml` is a **separate, opt-in** compose file holding SonarQube, Prometheus, Grafana, and the one-shot scanner containers (semgrep/trivy/gitleaks/snyk).

Why separate from the main `docker-compose.yml`?

The platform constitution (`.specify/memory/constitution.md` Principle I “Container-First Architecture”) defines the product topology as exactly **two containers**: `qbittorrent` and `download-proxy`, plus one optional host process (`webui-bridge.py`). Adding SonarQube, Postgres (for Sonar), Prometheus, Grafana, and four scanner images would silently extend that topology from 2 to 8 runtime services, which would violate the principle.

Keeping the quality services in a **separate compose file** means:

1. The product is still a two-container deploy. `./start.sh` launches only those two.
2. Quality tooling is explicit and scope-limited: `-f docker-compose.yml -f docker-compose.quality.yml` only when the developer wants it.
3. Profiles (`quality`, `run-once`, `observability`, etc) gate each service so nothing starts by accident.
4. The constitution does not need an amendment for quality tooling, since no product service depends on it.

See `docs/SCANNING.md` for scanner usage and `docs/OBSERVABILITY.md` for the Prometheus/Grafana story.

Profiles

Profile	Services	When to use
<code>quality</code>	<code>sonarqube</code> , <code>sonar-db</code>	Continuous quality gate host
<code>sonarqube</code>	<code>sonarqube</code> , <code>sonar-db</code>	Alias for above
<code>run-once</code>	<code>semgrep</code> , <code>trivy</code> , <code>gitleaks</code> , <code>snyk</code>	Scheduled / on-demand scans
<code>semgrep</code> , <code>trivy</code> , <code>gitleaks</code> , <code>snyk</code>	single scanner each	Target one tool
<code>observability</code>	<code>prometheus</code> , <code>grafana</code>	Metric collection + dashboards

Environment variables

<code>SONAR_DB_USER=sonar</code>	<code># default</code>
<code>SONAR_DB_PASSWORD=sonar</code>	<code># default “ override in production</code>
<code>SONAR_TOKEN=“</code>	<code># CI upload</code>
<code>SONAR_HOST_URL=http://localhost:9000</code>	
<code>SNYK_TOKEN=“</code>	<code># optional</code>
<code>GRAFANA_USER=admin</code>	<code># default</code>
<code>GRAFANA_PASSWORD=admin</code>	<code># default “ override in production</code>

Ports

All quality services bind to `127.0.0.1`: only so they are not exposed to the LAN:

- 9000 SonarQube UI
- 9090 Prometheus
- 3000 Grafana

Lifecycle

```
# Bring up SonarQube (~2 min cold start)
$COMPOSE_CMD -f docker-compose.yml -f docker-compose.quality.yml \
  --profile sonarqube up -d

# Run observability stack
$COMPOSE_CMD -f docker-compose.yml -f docker-compose.quality.yml \
  --profile observability up -d

# Stop quality services (product stays running)
$COMPOSE_CMD -f docker-compose.quality.yml --profile quality down

./scripts/scan.sh wraps these lifecycle commands for scanner one-shots.
```